

Integrarea Bazelor de Date

Socketuri în Java

Un socket reprezintă un mecanism de comunicare între două aplicații care rulează în rețea. Prin intermediul socketurilor, aplicațiile pot transmite și primi date folosind protocolul TCP/IP.

În Java, socketurile sunt utilizate frecvent pentru:

- aplicații client-server;
- chat-uri;
- transfer de fișiere;
- jocuri multiplayer;
- servicii web.

Tipuri de socketuri în Java

a) Socket

Clasa Socket este utilizată de client pentru conectarea la un server.

Exemplu:

```
Socket socket = new Socket("localhost", 5000);
```

Parametri:

- "localhost" → adresa serverului;
- 5000 → portul de comunicare.

b) ServerSocket

Clasa ServerSocket este utilizată de server pentru a asculta conexiuni de la clienți.

Exemplu:

```
ServerSocket serverSocket = new ServerSocket(5000);  
Socket client = serverSocket.accept();
```

Metoda accept():

- blochează execuția până când un client se conectează;
- returnează un obiect Socket.

Fluxuri pentru preluarea datelor asociate unui socket

Comunicarea prin socketuri se realizează prin fluxuri de intrare și ieșire.

Flux de intrare

Permite citirea datelor primite.

```
InputStream in = socket.getInputStream();
```

Flux de ieșire

Permite trimiterea datelor.

```
OutputStream out = socket.getOutputStream();
```

Exemplu simplu client-server

Server

```
import java.io.*;  
import java.net.*;  
  
public class Server {  
    public static void main(String[] args) throws Exception {  
  
        ServerSocket server = new ServerSocket(5000);  
  
        System.out.println("Server pornit...");  
    }  
}
```

```
Socket socket = server.accept();

BufferedReader reader =
    new BufferedReader(
        new InputStreamReader(socket.getInputStream()));

String mesaj = reader.readLine();

System.out.println("Mesaj primit: " + mesaj);

socket.close();
server.close();
}
}
```

Client

```
import java.io.*;
import java.net.*;

public class Client {
    public static void main(String[] args) throws Exception {

        Socket socket = new Socket("localhost", 5000);

        PrintWriter writer =
            new PrintWriter(socket.getOutputStream(), true);

        writer.println("Salut de la client!");

        socket.close();
    }
}
```

JAVA DATABASE CONNECTIVITY (JDBC)

JDBC (Java Database Connectivity) este o tehnologie Java care permite conectarea aplicațiilor Java la baze de date.

Cu ajutorul JDBC putem:

- executa interogări SQL;
- insera date;
- modifica date;
- șterge date;
- prelua rezultate din baza de date.

JDBC oferă independență față de sistemul de gestiune al bazei de date:

- MySQL;
- PostgreSQL;
- Oracle;
- SQL Server etc.

Arhitectura JDBC

Arhitectura JDBC este alcătuită din următoarele componente:

a) Aplicația Java

Programul scris de dezvoltator.

b) JDBC API

Set de clase și interfețe Java pentru conectarea la baze de date.

c) JDBC Driver Manager

Gestionează driverele JDBC.

d) Driver JDBC

Componentă specifică bazei de date.

Exemple:

- MySQL Connector/J;
- Oracle JDBC Driver.

e) Baza de date

+

Reprezentare schematică

Aplicatie Java

|

v

JDBC API

|

v

DriverManager

|

v

JDBC Driver

|

v

Baza de date

Etapele realizării unei aplicații folosind JDBC

Etapa 1 – Importul pachetelor

```
import java.sql.*;
```

Etapa 2 – Încărcarea driverului

Exemplu MySQL:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Etapa 3 – Stabilirea conexiunii

```
Connection conn =  
    DriverManager.getConnection(
```

```
"jdbc:mysql://localhost:3306/firma",  
"root",  
"parola");
```

Etapa 4 – Crearea unui obiect Statement

```
Statement stmt = conn.createStatement();
```

Etapa 5 – Executarea comenzilor SQL

```
ResultSet rs =  
    stmt.executeQuery("SELECT * FROM angajati");
```

Etapa 6 – Prelucrarea rezultatelor

```
while(rs.next()) {  
    System.out.println(rs.getString("nume"));  
}
```

Etapa 7 – Închiderea resurselor

```
rs.close();  
stmt.close();  
conn.close();
```

Interfața Statement

Interfața Statement este utilizată pentru executarea comenzilor SQL simple.

Creare

```
Statement stmt = conn.createStatement();
```

Metode importante

executeQuery()

Execută instrucțiuni SELECT.

```
ResultSet rs =  
    stmt.executeQuery("SELECT * FROM produse");
```

executeUpdate()

Execută:

- INSERT;
- UPDATE;
- DELETE.

```
int rows =  
    stmt.executeUpdate(  
        "DELETE FROM produse WHERE id=5");
```

execute()

Execută orice tip de comandă SQL.

```
stmt.execute("CREATE TABLE test(id INT);");
```

Dezavantaje ale Statement

- vulnerabil la SQL Injection;
- performanță mai redusă;
- dificil de utilizat cu parametri.

Interfața PreparedStatement

Ce este?

PreparedStatement reprezintă o versiune îmbunătățită a Statement.

Creare

```
PreparedStatement ps =  
    conn.prepareStatement(  
        "SELECT * FROM angajati WHERE salariu > ?");
```

Setarea parametrilor

```
ps.setInt(1, 3000);
```

Executare

```
ResultSet rs = ps.executeQuery();
```

Exemplu complet

```
PreparedStatement ps =  
    conn.prepareStatement(  
        "INSERT INTO angajati(nume, salariu) VALUES (?, ?)");  
  
ps.setString(1, "Ion");  
ps.setInt(2, 4500);  
  
ps.executeUpdate();
```

Avantaje principale

Securitate

Protejează împotriva atacurilor SQL Injection.

Performanță

Instrucțiunea SQL este compilată o singură dată.

Claritate

Cod mai ușor de întreținut.

Interfața CallableStatement

Ce este?

CallableStatement este utilizată pentru apelarea:

- procedurilor stocate;
- funcțiilor stocate;
din baza de date.

Creare

```
CallableStatement cs =  
    conn.prepareCall("{call adauga_angajat(?, ?)}");
```

Setarea parametrilor

```
cs.setString(1, "Maria");  
cs.setInt(2, 5000);
```

Executare

```
cs.execute();
```

Exemplu de procedură stocată

Procedură SQL

```
CREATE PROCEDURE adauga_angajat(  
    IN p_nume VARCHAR(50),  
    IN p_salariu INT  
)  
BEGIN  
    INSERT INTO angajati(nume, salariu)
```

```
VALUES(p_nume, p_salariu);  
END;
```

Apel din Java

```
CallableStatement cs =  
    conn.prepareCall("{call adauga_angajat(?, ?)}");  
  
cs.setString(1, "Ana");  
cs.setInt(2, 6000);  
  
cs.execute();
```

Diferențe între Statement, PreparedStatement și CallableStatement

Caracteristică	Statement	PreparedStatement	CallableStatement
Parametri	Nu	Da	Da
Protecție SQL Injection	Nu	Da	Da
Performanță	Redusă	Bună	Bună
Proceduri stocate	Nu	Nu	Da
Utilizare principală	SQL simplu	SQL parametrizat	Proceduri SQL

Exercitii

1. Creează baza de date:

universitate

și tabela:

studenti

cu următoarele câmpuri:

Câmp	Tip
id	INT PRIMARY KEY AUTO_INCREMENT
nume	VARCHAR(50)
varsta	INT
medie	DOUBLE

2. Creează un proiect Java.

Adaugă driverul JDBC MySQL.

Creează clasa:

TestConexiune

3. Inserează 3 studenți în tabel.
4. Afișează toți studenții din tabel.
5. Citește datele unui student de la tastatură și inserează-le în baza de date.
6. Caută studenții cu un anumit nume.
7. Mărește media studentului cu id-ul 1.
8. Șterge studentul cu id-ul 2.
9. Creează clasa Student în programul Java
10. Creează obiecte Student folosind ResultSet.
11. Creează un server care: ascultă pe portul 5000, afișează mesajele primite.
12. Trimite un mesaj către server
13. Trimite un mesaj automat de la server la orice mesaj primit
14. Creează tabela:
mesaje

Câmp	Tip
id	INT AUTO_INCREMENT
text_mesaj	VARCHAR(200)

Clientul trimite un mesaj.

Serverul:

- primește mesajul;
- îl salvează în baza de date.

Serverul:

- citește studenții din baza de date;
- trimite lista către client.

Clientul:

- afișează lista primită.